

LADiM: LAYERED DIAGNOSIS FOR MULTI-AGENT CODE MIGRATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Migrating deep learning code across frameworks is a practical requirement for reusing training programs in new software and hardware environments. Framework differences can change how a translated program computes gradients and updates its parameters without causing runtime errors. To address this problem, we propose LaDiM, a multi-agent framework that uses dependencies between training computations to guide diagnosis and repair. Starting from an initial translation, the Verifier Agent investigates where the target program diverges from the source and passes supporting code observations and test results to a separate Repair Agent. The Orchestrator returns measurements after each submitted change to guide further repair. Repository context management supports repair across files by tracking dependencies and retaining relevant evidence. Experiments across frameworks and programming languages demonstrate LaDiM’s effectiveness, with token savings of at most 57.4% over state-of-the-art methods at matched migration acceptance.

1 INTRODUCTION

Migrating deep learning code allows existing models and training programs to run in new software and hardware environments. Differences in operator semantics, numerical precision, and random number handling can change the computations performed after migration. Automatic differentiation and optimizer interfaces introduce further differences in how a model learns (Paszke et al., 2019; MindSpore, 2020). Large language models (LLMs) reduce the manual effort involved in code translation (He et al., 2025; Macedo et al., 2025b), while automated tests and interactive repair help assess and correct translated behavior (Rozière et al., 2022; Yang et al., 2024; Ibrahimzada et al., 2026). Training programs require these capabilities to preserve the computations that determine each parameter update.

A translated program can produce a loss close to that of the source while computing incorrect gradients. An optimizer error can also alter the parameter update without changing the current loss or gradients. We use semantic drift to describe these changes in training behavior caused by migration. Training computations have dependencies that help investigate such errors. A forward error may change gradients and updates, while an incorrect update may affect later forward computations. Locating the cause requires examining how the translated code produces the observed differences. This investigation also affects migration cost. Figure 1 compares complete token use on common migration tasks, where LaDiM achieves full acceptance with less than half the tokens used by MatchFixAgent.

We introduce LaDiM (Layered Diagnosis for Multi-Agent Code Migration), which uses dependencies between training computations to guide investigation of an initial translation. The Verifier Agent inspects code and runs tests to establish where source and target behavior diverge. For exam-

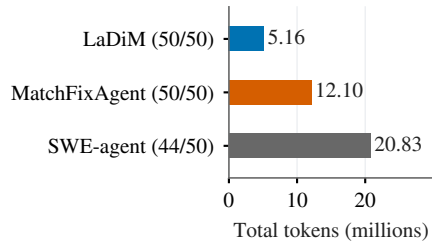


Figure 1: MindSpore migration costs, including initial translation and all subsequent calls. Labels give accepted tasks out of 50.

ple, matching forward values with missing gradients directs attention to parameter registration and the differentiation path. The agent records the code and measurements supporting its hypotheses and passes this evidence to a separate Repair Agent. After a change is submitted, the Orchestrator runs the acceptance checks and returns new measurements to guide subsequent attempts. This feedback connects each correction to the behavior it changes and exposes the next computation to investigate.

For repositories, migration must preserve shared implementations across multiple entry points. A change that corrects one caller can alter the behavior of another, and investigation may need evidence collected earlier in a different file. LaDiM tracks dependencies between work units and invalidates checks affected by changes. It retains observations and hypotheses in a retrievable archive and rebuilds the working context when the investigation moves between units. Together, these operations support repair across files while preserving the relationship between code changes and their measured effects.

We evaluate migration acceptance and model use from common initial translations under a shared protocol. LaDiM’s token savings cover both repair of faulty translations and confirmation of candidates that already pass the checks. On a time series repository, all three evaluated agents complete migration, with LaDiM using fewer tokens and repair calls. On a recommendation repository, LaDiM achieves the highest behavioral coverage and passes all ten original tests. Training signal and component ablations examine how observations and agent organization affect repair, while JAX and Java-to-Python experiments establish effectiveness across frameworks and languages. Our contributions are as follows.

- We formulate training-code migration around execution, forward values, gradients, and parameter updates, using their dependencies to investigate semantic faults.
- We develop LaDiM with independent investigation, evidence handoff, and verification after repair, together with dependency and context management for repositories.
- We evaluate migration across frameworks and languages, comparing acceptance and total token use and analyzing repair behavior, training signals, and agent components.

2 RELATED WORK

Code Translation and Framework Migration. Neural translation methods learn code mappings from multilingual corpora and use automated tests or compiler representations to improve translations (Lachaux et al., 2020; Rozière et al., 2022; Szafraniec et al., 2023). ExeCoder encodes functional semantics, syntax structure, and variable dependencies (He et al., 2025), while InterTrans explores intermediate translation paths (Macedo et al., 2025b). Framework migration tools such as MindConverter supply mappings between programming interfaces (MindSpore, 2023). RepoTransAgent uses repository context, test failures, and reflection to refine translated code (Guan et al., 2025). LaDiM investigates how a translated training program computes and updates its state. At repository scale, it coordinates changes to shared interfaces and checks their effect across declared entry points.

Differential Testing and Deep Learning Verification. Differential testing exposes compiler and program faults through disagreement between related executions (Yang et al., 2011; Le et al., 2014). DeepXplore and DeepGauge investigate behavioral differences and test coverage in deep learning systems (Pei et al., 2017; Ma et al., 2018). More directly related to framework migration, TensorScope tests corresponding APIs across frameworks to identify inconsistent behavior (Deng et al., 2023). Nablafuzz compares execution scenarios to test automatic differentiation, including first-order and higher-order gradients (Yang et al., 2023). LaDiM uses training differences as evidence for code repair. Its Verifier Agent relates the measured forward values, gradients, and parameter updates to the implementation, develops hypotheses about their origin, and supplies evidence that the Repair Agent can test when selecting a change.

Automated Program Repair and LLM Agents. Automated program repair uses tests or semantic constraints to guide patch generation (Nguyen et al., 2013), and pretrained language models generate patches from code and failure evidence (Xia et al., 2023). Interactive agents combine repository navigation, editing, and execution (Yang et al., 2024; Shinn et al., 2023; Chen et al., 2024b; Wang

et al., 2024). Multi-agent systems distribute development activities among specialized roles (Qian et al., 2024; Hong et al., 2024; Wu et al., 2024; Chen et al., 2024a). MatchFixAgent takes a source program and an existing translation and uses several semantic analyses to guide test generation and repair (Ibrahimzada et al., 2026). LaDiM includes initial translation and organizes diagnosis around dependencies in training computations. The independent investigation and its supporting observations pass to a separate Repair Agent, which uses measurements after each submission to revise its hypotheses. The comparison evaluates both methods from the same initial translation and includes translation costs.

3 METHODS

LaDiM uses dependencies between training computations to guide the investigation of a translated program. Starting from the Translator’s candidate, the Verifier Agent examines the observed discrepancies and tests the code that could explain them. Its findings and supporting observations pass to a separate Repair Agent, which uses this evidence to select and evaluate changes. The Orchestrator returns measurements after each submission, allowing repair to follow the errors revealed as execution progresses. Figure 2 shows this process.

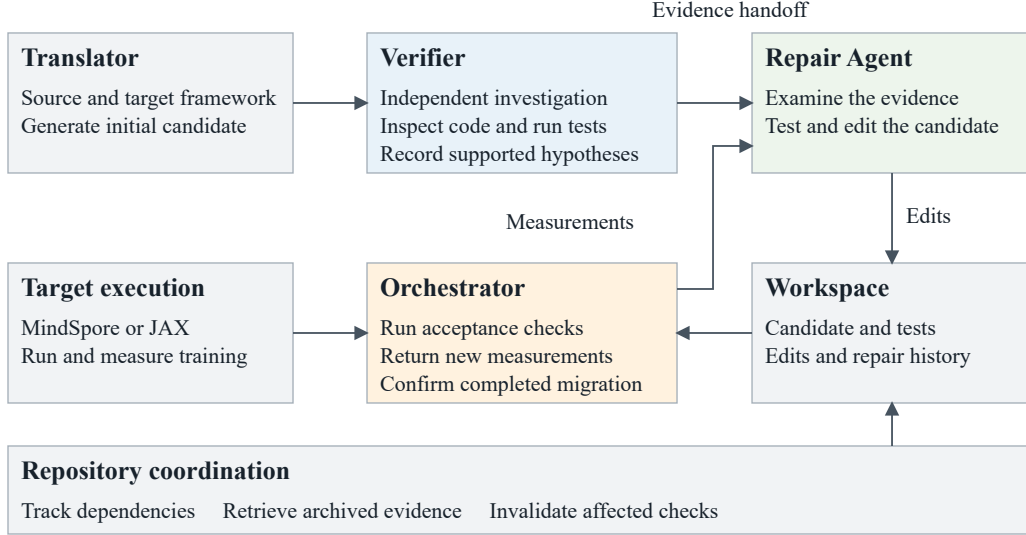


Figure 2: LaDiM’s migration workflow. The Translator generates a candidate for the Verifier Agent to investigate through code and training observations. The Repair Agent receives the evidence and revises the candidate. The Orchestrator checks submitted changes and returns measurements for subsequent attempts. Execution adapters obtain observations from the target framework.

3.1 PRELIMINARY

A migration task $\mathcal{T} = (P, L_s, F_s, L_t, F_t, W)$ specifies a source program P , its language and framework (L_s, F_s) , the requested target environment (L_t, F_t) , and an editable file scope W . Migration produces a target program $\hat{P}$ that preserves the source’s training behavior. An edit is one successfully applied editing operation, which can replace several code blocks across files in W . A submission follows a repair stage containing code inspection, tests, and edits.

At training step t , source and target executions produce corresponding observations $(e_t, v_t, g_t, \theta_{t+1} - \theta_t)$ and $(\hat{e}_t, \hat{v}_t, \hat{g}_t, \hat{\theta}_{t+1} - \hat{\theta}_t)$. Here e_t records the execution outcome, including success or an exception, v_t contains selected forward values and losses, g_t contains gradients, and θ_t denotes the parameters before the update. After aligning corresponding tensors and parameters,

we define

$$d_{\text{execution},t} = \mathbf{1}[e_t \neq \hat{e}_t], \quad d_{\text{forward},t} = v_t - \hat{v}_t, \quad (1)$$

$$d_{\text{gradient},t} = g_t - \hat{g}_t, \quad d_{\text{update},t} = (\theta_{t+1} - \theta_t) - (\hat{\theta}_{t+1} - \hat{\theta}_t). \quad (2)$$

For a numerical comparison $d = x - \hat{x}$, $d \simeq 0$ denotes agreement within its tolerance, such as $|d_j| \leq \alpha + \beta|x_j|$ for absolute tolerance α and relative tolerance β , or a specified norm bound. These discrepancies follow the training computation. Forward values determine the loss, differentiation produces the gradients, and the optimizer uses those gradients and its state to update parameters. The updated parameters then affect subsequent forward computations.

The LLM M performs translation, investigation, and repair under the role instructions A_T , A_V , and A_R , respectively. The Orchestrator $\mathcal{O}$ evaluates candidates and manages the shared investigation and repair budget B . In the algorithms, S holds the current candidate and observations, q is its verification status, H is a conversation, and C manages retained evidence, preserving the conversation for individual programs and coordinating context for repositories. Each $\text{QUERY}(M, A, H, b, B)$ invokes M with role instructions A and conversation H , charging its usage to the stage allowance b and shared budget B . The initial $\text{TRANSLATE}(M, A_T, \mathcal{T})$ call uses the same LLM to generate a candidate. Tools return observations and update S when an edit succeeds.

3.2 VERIFIER AGENT: LAYERED DIAGNOSIS

The Verifier Agent investigates the execution, forward, gradient, and update observations under A_V , which asks it to support hypotheses with code locations and distinguishing tests. Dependencies between these computations help narrow the causes. An **execution discrepancy** ($d_{\text{execution},t} = 1$) points to the exception and implicated calls. A **forward discrepancy** ($d_{\text{forward},t} \neq 0$) motivates tracing the operators that produce different values. When forward values agree, a **gradient discrepancy** ($d_{\text{gradient},t} \neq 0$) directs attention to parameter registration and differentiation. When forward values and gradients agree, an **update discrepancy** ($d_{\text{update},t} \neq 0$) points to the optimizer’s rule and state. The first affected step helps distinguish these causes because updates influence later forward values. The LLM selects code inspections and tests from this evidence. Algorithm 1 records the investigation and defines the shared model-call and tool-execution step. During diagnosis, production files are read-only and scratch tests remain editable.

Algorithm 1 Layered Diagnosis

```

1: function LAYEREDIAGNOSIS( $\mathcal{T}, S, M, B, C$ )
2:    $H \leftarrow \text{STARTCONVERSATION}(A_V, \mathcal{T}, S)$ ;  $b \leftarrow \text{DIAGNOSISALLOWANCE}(B)$ 
3:   while WITHINLIMITS( $b, B$ ) do
4:      $(H, a, R) \leftarrow \text{AGENTSTEP}(M, A_V, H, S, b, B, C)$ 
5:     if  $a.\text{complete}$  then break
6:   return  $\text{PUBICEVIDENCE}(H, C.\text{archive})$ 
7: function AGENTSTEP( $M, A, H, S, b, B, C$ )
8:    $H \leftarrow C.\text{BEFORECALL}(H, S)$ ;  $a \leftarrow \text{QUERY}(M, A, H, b, B)$ 
9:    $H \leftarrow H \parallel a$ ;  $R \leftarrow []$ 
10:  for all  $c \in a.\text{toolCalls}$  do
11:     $r \leftarrow \text{EXECUTE}(c, S, \text{productionReadOnly} = (A = A_V))$ 
12:     $H \leftarrow H \parallel (c, r)$ ;  $R \leftarrow R \parallel (c, r)$ ;  $C.\text{RECORD}(c, r, S)$ 
13:  return  $(H, a, R)$ 

```

3.3 REPAIR AGENT: EVIDENCE HANDOFF

The Repair Agent starts a separate conversation with the Verifier Agent’s code observations, test results, and diagnosis, attributed to the Verifier Agent. It can reassess these findings, run further tests, and edit files in W . For individual programs, the Orchestrator refreshes the measurements after diagnosis and confirms candidates that pass the public checks. Unresolved discrepancies enter the repair loop in Algorithm 2 with the diagnosis evidence E , current observations $S.m$, and verification status q .

The repair loop reuses `AGENTSTEP` and its ordered tool results R . The stage flags F record whether a production edit succeeded and whether a test ran after the latest edit. A final response ends the stage when both conditions hold; otherwise, the agent receives the pending action and continues within its allowance. After each submission, the Orchestrator measures the candidate and returns the unresolved differences. The candidate and repair conversation persist across attempts under the shared budget.

Algorithm 2 `REPAIR($\mathcal{T}, S, E, q, M, B, C$)

---`

```

1:  $H \leftarrow \text{STARTCONVERSATION}(A_R, \mathcal{T}, \text{AttributeToVerifier}(E))$ 
2:  $C.\text{handoff} \leftarrow \text{true}$ 
3: while SUBMISSIONSLEFT( $B$ )  $\wedge$  BUDGETLEFT( $B$ ) do
4:    $H \leftarrow H \parallel S.m$ ;  $F \leftarrow (\text{false}, \text{false})$ ;  $b \leftarrow \text{REPAIRALLOWANCE}(B)$ 
5:   while WITHINLIMITS( $b, B$ ) do
6:      $(H, a, R) \leftarrow \text{AGENTSTEP}(M, A_R, H, S, b, B, C)$ ;  $F \leftarrow \text{UPDATEFLAGS}(F, R)$ 
7:     if  $a.\text{complete} \wedge F.\text{edited} \wedge F.\text{tested}$  then break
8:     if  $a.\text{complete}$  then  $H \leftarrow H \parallel \text{PENDINGACTION}(F)$ 
9:      $(S.m, q) \leftarrow \mathcal{O}.\text{VERIFY}(S.\hat{P})$ ;  $B.\text{submissions} \leftarrow B.\text{submissions} + 1$ 
10:    if  $q = \text{accepted}$  then
11:      return  $(S.\hat{P}, q)$ 
12: return  $(S.\hat{P}, q)$ 

```

3.4 REPOSITORY COORDINATION

A change to a shared implementation can affect several callers in a repository. LaDiM provides a map of files, imports, symbols, and notebook cells from which the agent proposes work units with file scopes, interface goals, and dependencies. It selects a unit whose prerequisites have current checkpoints, then investigates the related interfaces. Checkpoints record syntax validation and selected local tests. Code changes invalidate checkpoints for affected units and their dependents, prompting checks of the callers that may have changed behavior. The Orchestrator evaluates the complete repository, including its entry points and training computations, to determine acceptance.

Repository context management carries relevant evidence as investigation moves across files. LaDiM archives observations and conversations and provides a tool for retrieving detailed records. At the role handoff, an actual switch between work units, or a context capacity boundary, it rebuilds the conversation around the current plan, hypotheses, measurements, and relevant code already inspected. Earlier measurements retain their association with the candidate on which they were obtained. Algorithm 3 starts with complete repository verification and returns an accepted candidate immediately. Otherwise, the initial measurements support diagnosis, and the resulting evidence starts the repair loop. Its context operations serve Algorithms 1 and 2.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Tasks. The main comparison contains 50 PyTorch-to-MindSpore migration tasks covering tensor operations, CNNs, MLPs, Transformers, language models, and parameter and optimizer interfaces. Table 1 summarizes this collection and the supplied candidates used to study repair and training signals. Across languages, we migrate 18 Java/DJL programs to Python/PyTorch, comprising eight textbook examples and ten applications (DJL Contributors, 2026b;a). Their sources average 221.2 nonblank lines. The public time series (jinglescode, 2026) and recommendation (gauravchak, 2026) repositories contain 13 and 24 code files or notebooks, totaling 865 and 2,086 nonblank lines, including original tests and notebook code cells.

Evaluation protocol. Methods use common source programs, evaluation inputs, and corresponding initial parameters. Repair methods also share initial translations. Acceptance requires all applicable comparisons of execution, forward values, gradients, parameter updates, and specified optimizer state, together with shape, data type, and trainability checks. In the main migration comparisons,

Algorithm 3 Repository migration and context operations

```

1: function REPOSITORYMIGRATE( $\mathcal{T}, M, B$ )
2:    $S.\hat{P} \leftarrow \text{TRANSLATE}(M, A_T, \mathcal{T}); C \leftarrow \text{REPOSITORYCONTEXT}(\text{Map}(\mathcal{T}))$ 
3:    $(S.m, q) \leftarrow \mathcal{O}.\text{VERIFY}(S.\hat{P})$ 
4:   if  $q = \text{accepted}$  then
5:     return  $(S.\hat{P}, q)$ 
6:    $E \leftarrow \text{LAYEREDIAGNOSIS}(\mathcal{T}, S, M, B, C)$ 
7:   return REPAIR( $\mathcal{T}, S, E, q, M, B, C$ )
8: function C.RECORD( $c, r, S$ )
9:    $C.\text{archive} \leftarrow C.\text{archive} \parallel (c, r)$ 
10:  if CODECHANGED( $r$ ) then
11:    MARKHISTORICAL( $C.\text{measurements}$ )
12:    INVALIDATE( $C.\text{checkpoints}, \text{AffectedUnitsAndDependents}(r, C.\text{plan})$ )
13:     $(C.\text{plan}, C.\text{unit}, C.\text{checkpoints}) \leftarrow \text{UPDATE}(c, r, C)$ 
14: function C.BEFORECALL( $H, S$ )
15:  if  $C.\text{handoff} \vee \text{UNITSWITCHED}(C) \vee \text{ATCAPACITY}(H)$  then
16:    ARCHIVE( $C, H$ )
17:     $K \leftarrow \text{CURRENTREADCODE}(C.\text{unit}, S.\hat{P})$ 
18:     $E \leftarrow \text{PUBLICFINDINGS}(H, C.\text{archive})$ 
19:     $H \leftarrow \text{REBUILD}(H.\text{role}, C.\text{plan}, E, S.m, K)$ 
20:     $C.\text{handoff} \leftarrow \text{false}; C.\text{previousUnit} \leftarrow C.\text{unit}$ 
21:  return  $H$ 

```

Table 1: Program collections for migration and repair, with their candidate construction, model coverage, and task counts.

Collection	Candidate construction	Model or operation coverage	Tasks
PyTorch to MindSpore	Generated translations	Operators, CNNs, MLPs, Transformers, language models	50
Natural translation faults	Supplied translations	Recurrent, convolutional, attention, and set models	10
Training-signal ablation	Controlled training faults	CNN, image MLP, Transformer, causal language model	16
JAX repair	Supplied faulty candidates	MLP and CNN	6

individual programs retain their own state over two consecutive steps, and repositories use three. Candidates that pass the public checks undergo confirmation on additional seeds. Main MindSpore absolute/relative tolerances are $10^{-4}/10^{-4}$ for forward values, $10^{-4}/10^{-3}$ for gradients, and $10^{-5}/10^{-3}$ for updates. BF16 matrix multiplication uses absolute tolerance 10^{-2} . Appendix A gives the settings for each study.

The repository tables report *behavior checks passed*, counting numerical comparisons, execution, and source behavior. Time series has 69 checks across public and confirmation evaluations. Recommendation has 145 public checks, comprising 114 numerical, 24 execution and differentiation, and seven source-behavior checks. Its ten original tests are reported separately. Complete repository acceptance also requires the declared entry points, file coverage, and documentation checks.

We retain the baselines’ algorithms, prompts, tools, parsers, and retry policies, using the same DeepSeek backend. CodeTransEngine uses its direct translation configuration. Individual migration allows 40 investigation and repair calls, 120,000 output tokens, 1,800 seconds, and four external submissions. Repository limits are 80 calls, 480,000 output tokens, 3,600 seconds, and four submissions. A submission can follow several model calls and edits. Migration costs include the common initial translation once and all subsequent input and output tokens, including failed at-

Table 2: Migration results and model use on common inputs. Panels (a,b) report acceptance and complete token costs. Panels (c,d) report repository checks and costs. A dash denotes a method without LLM calls.

Method	Model calls	Input tokens (millions)	Output tokens (millions)	Total tokens (millions)	Accepted	
(a) PyTorch to MindSpore						
Direct LLM	29	0.018	0.411	0.429	29/50	
CodeTransEngine (Macedo et al., 2025a)	29	0.022	0.257	0.280	31/50	
MSAdapter (OpenI, 2025)	–	–	–	–	15/50	
SWE-agent (Yang et al., 2024)	1,097	19.845	0.987	20.831	44/50	
MatchFixAgent (Ibrahimzada et al., 2026)	619	10.750	1.347	12.097	50/50	
LaDiM (ours)	326	4.298	0.861	5.159	50/50	
(b) Java/DJL to Python/PyTorch						
Direct LLM	18	0.142	0.552	0.694	1/18	
InterTrans (Macedo et al., 2025b)	89	0.680	1.606	2.286	1/18	
Test-guided repair	71	1.594	2.881	4.475	7/18	
SWE-agent	682	23.063	1.258	24.321	8/18	
MatchFixAgent	524	26.335	2.020	28.355	8/18	
LaDiM (ours)	345	19.470	2.009	21.479	9/18	
(c) Time series repository						
Method	Behavior checks passed	Gradients	Parameter updates	Entry points	Repair calls	Total tokens (millions)
SWE-agent	69/69	9/9	9/9	3/3	80	8.200
MatchFixAgent	69/69	9/9	9/9	3/3	69	13.598
LaDiM (ours)	69/69	9/9	9/9	3/3	35	2.614
(d) Recommendation repository						
Method	Behavior checks passed	Gradients	Parameter updates	Original tests	Repair calls	Total tokens (millions)
SWE-agent	104/145	17/18	12/18	10/10	80	8.317
MatchFixAgent	129/145	18/18	16/18	0/10	80	8.679
LaDiM (ours)	130/145	18/18	15/18	10/10	80	8.811

MatchFixAgent passes 0/10 original recommendation tests because an unmigrated PyTorch import prevents test collection.

tempts. Supplied-candidate studies report repair costs. Failures remain in the task denominator, and acceptance at each budget scores the current candidate.

4.2 MAIN RESULTS

Migration to MindSpore. LaDiM completes all 50 migrations with less than half the total tokens used by MatchFixAgent (Table 2a). Its savings extend to both faulty translations and programs that are already correct. On faulty inputs, LaDiM uses training discrepancies to investigate the code and guide corrective edits. On initially correct inputs, investigation and confirmation establish acceptance without corrective changes. Figure 3 separates these costs and shows lower token use on 27 of the 29 distinct inputs. LaDiM reaches 46/50 acceptance after one submission and 50/50 after two, retaining full acceptance at four. Seven of the nine faulty inputs are repaired at the first submission, and new measurements guide the remaining two corrections. The complete process uses 326 model calls, compared with MatchFixAgent’s 619, with reduced input usage accounting for 93.0% of the token savings.

Migration across languages. LaDiM completes all eight textbook examples and one application in the Java/DJL collection, covering MatchFixAgent’s accepted set and an additional recurrent network example (Table 2b). All repair methods preserve the initially accepted application. The added coverage comes from training examples where corresponding interfaces permit comparison of the

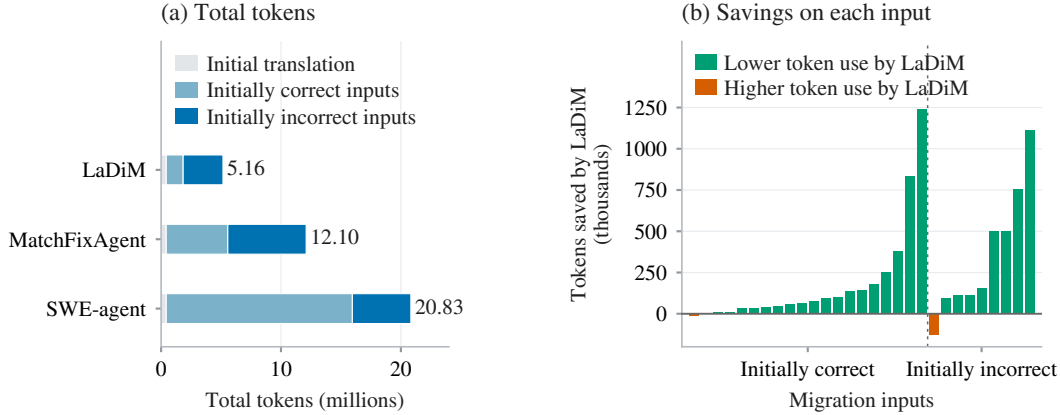


Figure 3: Token use on the common MindSpore collection. (a) Totals include initial translation and subsequent calls grouped by initial correctness. (b) Each bar shows MatchFixAgent tokens minus LaDiM tokens for one input, sorted within each group. Positive values indicate lower LaDiM cost, occurring on 19 of 20 initially correct and 8 of 9 initially incorrect inputs.

training computations. LaDiM achieves the highest acceptance with 11.7% fewer tokens than SWE-agent and 24.2% fewer than MatchFixAgent.

Repository migration. All three agents pass 69/69 checks and execute the three time series entry points, which share a model across a training script, notebook, and teaching examples. LaDiM completes repair in 35 calls, compared with 80 for SWE-agent and 69 for MatchFixAgent, using 68.1% and 80.8% fewer total tokens. This result extends the efficiency advantage to changes that must work across several callers.

On recommendation, LaDiM achieves the highest behavioral coverage, passing 130/145 checks and all ten original tests. Its repaired implementation supports reward model inference and retrieval, which remain unresolved in both baselines, and executes the training command. This coverage extends from the shared model computations to their use in training, inference, and retrieval, where repairs must work together across repository interfaces. Appendix E gives the detailed checks.

4.3 ANALYSIS EXPERIMENTS

Repairing Natural Translation Faults. LaDiM repairs four of five initially incorrect translations and preserves all five initially correct programs, yielding 9/10 final acceptance (Table 3). The baselines preserve the correct programs and leave all five faulty programs unresolved. An LSTM translation illustrates how successive measurements guide these corrections. After the first submission leaves an unregistered recurrent operator unresolved, LaDiM reproduces the failure, inspects registration, and adds the recurrence using target-framework operations. The second submission exposes recursive tensor indexing. A focused test identifies the dispatch cycle, and the third submission corrects it and passes loss, gradient, and update checks. As execution reaches further code, new measurements guide the next correction (Appendix D).

Table 3: Repair and preservation on ten supplied translations. Direct repair shares LaDiM’s inspection, testing, and editing tools. Costs include all investigation and repair calls.

Method	Initially incorrect programs repaired	Initially correct programs preserved	Repair tokens (millions)	Final accepted
Direct repair	0/5	5/5	3.751	5/10
SWE-agent	0/5	5/5	12.678	5/10
MatchFixAgent	0/5	5/5	14.351	5/10
LaDiM	4/5	5/5	13.207	9/10

Training Signals and Detection Latency. LaDiM’s gradient and update checks detect their respective faults at step 1 in all 12 runs per fault, while the mean loss-difference curves cross the detection threshold at steps 31 and 18 (Figure 4). These trajectories follow four models for 50 steps with source and target executions retaining separate state. Gradient faults can preserve current loss, and update faults can preserve forward values and gradients; their effects enter later forward computations through the changed training state. Individual loss trajectories cross within 50 steps in only 3/12 and 4/12 runs. Direct checks expose the affected computation as soon as it diverges. The comparisons use loss absolute difference, gradient norm absolute difference, and relative update L2 difference at thresholds 0.02, 0.05, and 0.03. With per-step synchronization, each target starts the checked step from the source state. Correct translations pass the recorded checks, with maximum loss difference 3.815×10^{-6} .

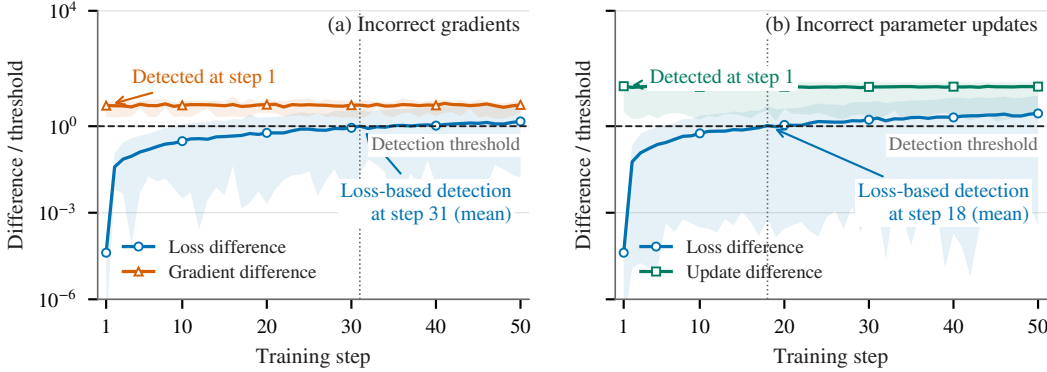


Figure 4: LaDiM detects gradient scaling (a) and partial update suppression (b) at step 1 in all 12 runs per panel through gradient and update discrepancies. Loss-based detection on the mean loss-difference curves occurs at steps 31 and 18. Individual loss trajectories cross within 50 steps in 3/12 and 4/12 cases. Differences are normalized by their thresholds; curves show means and shading shows ranges.

Generalization Across Frameworks. LaDiM repairs all six MLP and CNN candidates in JAX within the first submission (Table 4). The same diagnosis and repair procedure verifies JAX gradients and Optax updates, extending its application beyond MindSpore.

Table 4: Repair of six supplied JAX candidates with up to four submissions. Costs cover investigation and repair.

Method	Model calls	Repair tokens	Accepted
Direct repair (shared tools)	57	353,311	6/6
LaDiM	76	674,009	6/6

4.4 ABLATION STUDIES

Training signals. We compare cumulative feedback on 16 fixed candidates, with one execution, forward, gradient, and update fault in each CNN, image MLP, Transformer classifier, and small causal language model. Sources, candidates, tools, and budgets are fixed. Available feedback determines whether repair begins and stops. Table 5a reports initially undetected faults and final acceptance under complete verification. Each added signal exposes another fault class, increasing acceptance from 4/16 to 16/16. Forward values reveal changed computations, gradients reveal differentiation faults, and updates reveal optimizer faults.

Agent components. We cumulatively add a read-only Verifier investigation, independent evidence handoff, and repository context management to the Repair Agent on the time series repository (Table 5b). Each condition runs once with the same initial candidate, full training signals, base repair tools, and total budgets. The base condition retains the repair prompts and tool assistance. Investigation continues in the repair conversation until the independent handoff is added.

Table 5: Cumulative ablations. (a) Training signals on 16 faulty candidates; tokens cover investigation and repair. Initially undetected faults trigger no repair, and final acceptance uses complete verification. (b) Agent components on the time series repository; calls count investigation and repair, and tokens additionally include the common initial translation. Each row retains the preceding signals or components within its panel.

<i>(a) Training signals: 16 faulty candidates</i>			
Available feedback	Initially undetected faults	Total tokens (millions)	Final accepted
Execution and basic checks	12	2.089	4/16
+ Forward values	8	2.603	8/16
+ Gradients	4	3.303	12/16
+ Parameter updates	0	3.419	16/16
<i>(b) Agent components: time series repository</i>			
Components	Behavior checks passed	Investigation and repair calls	Total tokens (millions)
Repair Agent	69/69	26	6.128
+ Verifier investigation	69/69	18	3.896
+ Independent evidence handoff	69/69	26	6.047
+ Repository context management	69/69	22	1.937

All four conditions complete the migration and pass 69/69 behavior checks. The full method uses the fewest total tokens, 1.937 million, reducing cost by 68.4% relative to the base Repair Agent. The cost of an independent handoff depends on how its evidence is carried forward: adding repository context management reduces investigation and repair input tokens from 5.731 million to 1.700 million. Retaining current findings and relevant code allows the Repair Agent to continue from the investigation with less repeated context.

5 CONCLUSION

We introduced LaDiM, a multi-agent framework for migrating deep learning code while preserving training behavior. LaDiM uses dependencies between training computations to guide diagnosis and passes the resulting evidence to a separate Repair Agent. Verification after each submission guides subsequent changes, and dependency tracking with evidence retrieval supports repair across files. This design detects and repairs semantic errors that remain hidden when evaluation checks only execution or loss. Experiments across frameworks and programming languages demonstrate LaDiM’s effectiveness in preserving training behavior and its efficiency in diagnosis and repair.

AI USE STATEMENT

REFERENCES

- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=EHg5GDnyq1>.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=KuPixIqPiQ>.
- Zizhuang Deng, Guozhu Meng, Kai Chen, Tong Liu, Lu Xiang, and Chunyang Chen. Differential testing of cross deep learning framework APIs: Revealing inconsistencies and vulnerabilities. In *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 7393–7410. USENIX Association, 2023.

2023. URL <https://www.usenix.org/conference/usenixsecurity23/presentation/deng-zizhuang>.
- DJL Contributors. Dive into deep learning with java. <https://github.com/deepjavalibrary/d2l-java>, 2026a. Source repository, accessed September 21, 2026.
- DJL Contributors. Deep java library. <https://github.com/deepjavalibrary/djl>, 2026b. Source repository, accessed September 21, 2026.
- gauravchak. Two tower models. https://github.com/gauravchak/two_tower_models, 2026. Source repository, accessed September 21, 2026.
- Google. torchax: A PyTorch frontend for JAX. <https://google.github.io/torchax/>, 2025. Accessed: 2026-06-02.
- Ziqi Guan, Xin Yin, Zhiyuan Peng, and Chao Ni. RepoTransAgent: Multi-agent LLM framework for repository-aware code translation. *arXiv preprint arXiv:2508.17720*, 2025. doi: 10.48550/arXiv.2508.17720. URL <https://arxiv.org/abs/2508.17720>.
- Minghua He, Yue Chen, Fangkai Yang, Pu Zhao, Wenjie Yin, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. ExeCoder: Empowering large language models with executability representation for code translation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 7099–7125, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.362. URL <https://aclanthology.org/2025.emnlp-main.362/>.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- Ali Reza Ibrahimzada, Brandon Paulsen, Reyhaneh Jabbarvand, Joey Dodds, and Daniel Kroening. MatchFixAgent: Language-agnostic autonomous repository-level code translation validation and repair. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=MuyXpH3GL1>.
- jinglescode. Time series forecasting with pytorch. <https://github.com/jinglescode/time-series-forecasting-pytorch>, 2026. Source repository, accessed September 21, 2026.
- Marie-Anne Lachaux, Baptiste Rozière, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. In *Advances in Neural Information Processing Systems*, volume 33, pp. 20601–20611, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html>.
- Vu Le, Mehrdad Afshari, and Zhendong Su. Compiler validation via equivalence modulo inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 216–226. ACM, 2014. doi: 10.1145/2594291.2594334. URL <https://doi.org/10.1145/2594291.2594334>.
- Lei Ma, Felix Juefei-Xu, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Chunyang Chen, Ting Su, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. DeepGauge: Multi-granularity testing criteria for deep learning systems. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, pp. 120–131. ACM, 2018. doi: 10.1145/3238147.3238202. URL <https://doi.org/10.1145/3238147.3238202>.
- Marcos Macedo, Yuan Tian, and Bram Adams. CodeTransEngine: Ready-to-use backend for LLM-based code translation. In *ICLR 2025 Third Workshop on Deep Learning for Code*, 2025a. URL <https://openreview.net/forum?id=9WHJuzjPJf>.

- Marcos Macedo, Yuan Tian, Pengyu Nie, Filipe R. Cogo, and Bram Adams. InterTrans: Leveraging transitive intermediate translations to enhance LLM-based code translation. In *Proceedings of the 47th International Conference on Software Engineering*, pp. 2819–2831. IEEE, 2025b. doi: 10.1109/ICSE55347.2025.00236. URL <https://doi.org/10.1109/ICSE55347.2025.00236>.
- MindSpore. MindSpore: A new deep learning computing framework. <https://www.mindspore.cn/en>, 2020. Framework white paper and documentation.
- MindSpore. Migrating from third party frameworks with MindConverter. https://www.mindspore.cn/mindinsight/docs/en/r2.0.0-alpha/migrate_3rd_scripts_mindconverter.html, 2023. Accessed: 2026-06-02.
- Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. SemFix: Program repair via semantic analysis. In *Proceedings of the 35th International Conference on Software Engineering*, pp. 772–781. IEEE, 2013. doi: 10.1109/ICSE.2013.6606623. URL <https://doi.org/10.1109/ICSE.2013.6606623>.
- OpenI. MSAdapter. <https://openi.pcl.ac.cn/OpenI/MSAdapter/src/tag/v0.6.0>, 2025. Version 0.6.0. Accessed: 2026-09-20.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pp. 8024–8035, 2019. URL <https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>.
- Kexin Pei, Yinzhi Cao, Junfeng Yang, and Suman Jana. DeepXplore: Automated whitebox testing of deep learning systems. In *Proceedings of the 26th Symposium on Operating Systems Principles*, pp. 1–18. ACM, 2017. doi: 10.1145/3132747.3132785. URL <https://doi.org/10.1145/3132747.3132785>.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 15174–15186. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.810. URL <https://aclanthology.org/2024.acl-long.810/>.
- Baptiste Rozière, Jie M. Zhang, Francois Charton, Mark Harman, Gabriel Synnaeve, and Guillaume Lample. Leveraging automated unit tests for unsupervised code translation. In *Proceedings of the Tenth International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=cmt-6KtR4c4>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Marc Szafraniec, Baptiste Rozière, Hugh James Leather, Patrick Labatut, Francois Charton, and Gabriel Synnaeve. Code translation with compiler representations. In *Proceedings of the Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=XomEU3eNeSQ>.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 50733–50749, Vienna, Austria, 2024. PMLR. URL <https://proceedings.mlr.press/v235/wang24h.html>.

- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Proceedings of the First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=uAjxFFing2>.
- Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering*, pp. 1482–1494. IEEE, 2023. doi: 10.1109/ICSE48619.2023.00129. URL <https://doi.org/10.1109/ICSE48619.2023.00129>.
- Chenyuan Yang, Yinlin Deng, Jiayi Yao, Yuxing Tu, Hanchi Li, and Lingming Zhang. Fuzzing automatic differentiation in deep-learning libraries. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023. doi: 10.1109/ICSE48619.2023.00105. URL <https://arxiv.org/abs/2302.04351>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. doi: 10.48550/arXiv.2405.15793. URL <https://arxiv.org/abs/2405.15793>.
- Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 283–294. ACM, 2011. doi: 10.1145/1993498.1993532. URL <https://doi.org/10.1145/1993498.1993532>.

A ADDITIONAL EXPERIMENTAL PROTOCOLS

A.1 TASK CONSTRUCTION AND ACCOUNTING

The main MindSpore collection contains 50 task identifiers. Twelve entries reuse existing native source programs, and the remaining entries use programs authored from public interfaces and declared semantics. The identifiers correspond to 29 distinct combinations of source program and evaluation settings drawn from 24 source files. Conditions with identical source, evaluation settings, initial candidate, method, and inputs are executed once and associated with the corresponding identifiers. Acceptance in the main table uses all 50 identifiers, while token totals count actual calls. LaDiM, SWE-agent, and MatchFixAgent share the same initial translation for each distinct input. Production edits are restricted to the candidate program, with separate temporary tests available for investigation. Sources and task specifications remain fixed.

The comparison across languages contains ten DJL application programs and eight textbook examples. LaDiM accepts one application and all eight examples. SWE-agent and MatchFixAgent each accept the same application and seven examples, while test-guided repair accepts seven examples. The saved initial translations and their generation costs are shared by the repair conditions. Inter-Trans uses its direct Java-to-Python path and an intermediate JavaScript path, with native search and extraction procedures. Its 2,286,034 tokens include 574,304 from incomplete generation calls and 1,711,730 from the completed search. The final evaluation compares outputs, losses, gradients, parameter updates, and observable optimizer state over two consecutive training steps.

The common MindSpore initial translations cost 429,109 tokens. This amount is already included once in each repair method’s total. Additional LaDiM investigation uses 1,409,686 tokens on initially accepted inputs and 3,320,339 on initially faulty inputs. The corresponding MatchFixAgent totals are 5,135,863 and 6,532,453. Across the 29 distinct inputs, the median ratio of LaDiM to MatchFixAgent total tokens is 0.588. Shared initial translation costs cancel when their paired costs are subtracted. MatchFixAgent reaches full acceptance within its first external submission through its internal analysis and repair orchestration.

Source lengths count nonblank physical lines, including comments. Program means count each distinct source once. The 50 MindSpore identifiers use 24 source files, the signal study uses four, and the JAX study uses two. Repository lengths include all source Python files and notebook code cells, including original tests and excluding the external evaluation harness.

A.2 NUMERICAL COMPARISONS AND MIGRATION SETTINGS

The main MindSpore collection uses absolute/relative tolerances of $10^{-4}/10^{-4}$ for forward values, $10^{-4}/10^{-3}$ for gradients, and $10^{-5}/10^{-3}$ for updates. BF16 matrix multiplication uses absolute tolerance 10^{-2} . Shapes, data types, initial values, and trainability must match. Parameter and optimizer tasks test their specified enumeration, gradient assignment, or update behavior. The source program and task requirements are available to each method. Acceptance also checks that the target framework executes the required computations.

SWE-agent retains its native tools and controller. MatchFixAgent retains its analysis and repair orchestration through a shared DeepSeek tool backend. CodeTransEngine generates its own translation through the native direct path, and MSAdapter uses its native interface mapping. LaDiM allows eight calls per investigation or repair stage on individual programs. Agent calls allow at most 16,384 output tokens, while initial translation and the repair loop that rewrites the complete candidate allow at most 131,072 per call. The latter has four repair rounds. All diagnosis and repair stages share the cumulative limits in Section 4.1. Repository calls allow at most 32,768 output tokens. Output budgets include reasoning tokens, and all input usage is recorded.

A.3 RANDOM SEEDS AND INITIAL STATES

Individual migration. The main MindSpore and Java/DJL comparisons use seed 101 for public evaluation and seeds 202 and 303 for confirmation of a passing candidate. The MindSpore evaluator uses NumPy’s random generator to sample numerical inputs and training targets. Corresponding parameters are assigned identical initial values generated from a hash of each parameter name; these

values remain fixed across the three input seeds. Discrete inputs, masks, and prescribed optimizer-test values retain their task-specific settings. For Java/DJL programs, each paired evaluation supplies the original initial parameters and two preprocessed batches to the candidate for consecutive training steps. The component comparisons on these programs use the same settings.

Natural faults and training signals. All ten natural translation tasks use seed 42 for public evaluation and 1042 and 2042 for confirmation. The 16 training-signal candidates use 4101, 5101, and 6101, with 4101 serving as the public seed. The paired evaluator sets the Python, NumPy, and PyTorch random states before constructing the source model and copies its initial state to both executions. A separate CPU PyTorch generator seeded with $s + 101$ produces inputs and labels for evaluation seed s , including token inputs for text models. The signal study uses all three seeds when checking fault construction and scoring the final candidate with all signals. The additional repair collections retain their original unit probes and apply their task-specific acceptance checks without an additional multi-seed confirmation stage.

JAX repair. The MLP and CNN programs use public seed 6701 and confirmation seeds 6702 and 6703. The evaluator initializes PyTorch’s random state with the evaluation seed before constructing the source or candidate model. Inputs and class labels use a separate CPU generator with seed $s + 101$. TorchAX transfers the candidate to JAX, where native differentiation and Optax perform the checked training step.

Detection trajectories. Seeds 300, 301, and 302 define three repeated trajectories for each model. Each seed initializes the model’s floating-point parameters, and its initial state is shared across the source and target executions. Image batch order uses seed $s + 5000$, text-classification batch order uses $s + 6000$, and language-model window selection uses $s + 7000$. The fault and synchronization conditions for a given model and seed share the same initial state and 50-step batch sequence.

Time series repository. The main comparison and component ablation use public seed 101 and confirmation seeds 202 and 303. The source model is initialized with the evaluation seed, and its parameters are copied to the target. NumPy generators use seed $s + 47$ for the three training batches and seed s for the synthetic price series. The target MindSpore random state also uses s . The training script, notebook, and teaching examples are exercised in the public evaluation; confirmation repeats the numerical checks on the remaining two seeds.

Recommendation repository. The public numerical evaluation uses seed 101 for source model initialization and NumPy generation of user identifiers, features, histories, item identifiers, positions, and labels. The source parameters and retrieval corpus are then supplied to the target, whose MindSpore random state uses the same seed. The protocol assigns seeds 202 and 303 to confirmation after all public checks pass; the reported recommendation runs finish at the public evaluation. The original tests use seed 42, including Python, NumPy, and framework random states. For their exact-value history-encoder checks, the public task supplies the source parameters initialized with PyTorch seed 42. The one-epoch training command uses seed 101.

A.4 TRAINING SIGNAL AND FRAMEWORK STUDIES

The training signal and JAX studies use the same investigation and evidence handoff procedure with editing examples and format-correction feedback in their tool prompts. The signal experiment applies four controlled modifications to four model classes. The execution fault uses an invalid optimizer keyword, the forward fault adds a constant loss offset, the gradient fault scales gradients, and the update fault changes the learning rate. Each setting retains the initial investigation, current workspace, and cumulative call budget. Its visible checks determine whether repair starts and stops. Final acceptance uses all training signals on three seeds. MindSpore measurements use the `torch4ms` forward and backward bridge, followed by optimizer updates on its parameters.

The JAX study contains six faults in MLP and CNN candidate programs. TorchAX exposes the PyTorch interface over JAX (Google, 2025), with JAX computing gradients and Optax applying updates. Both LaDiM and direct repair allow four external submissions and accept every task at the first submission. LaDiM uses 674,009 tokens and direct repair uses 353,311. Native Ivy and `torch2jax` are also tested on these supplied faulty candidates and accept 0/6 repairs, while their healthy-input checks pass.

A.5 DETECTION MEASUREMENTS

The detection study uses four models, three seeds, and 50 training steps. Gradient scaling and partial update suppression are evaluated both with per-step synchronization to the source state and with each execution retaining its own state. Loss, gradient norm, and relative update thresholds are 0.02, 0.05, and 0.03. Figure 4 uses trajectories with separate state and reports the mean and range across 12 runs per fault. The loss threshold crossings at steps 31 and 18 refer to mean curves. Direct gradient or update checks detect all respective faults at step 1.

The scalar summaries are

$$\Delta_{\text{loss}} = |L(P) - L(\hat{P})|, \quad (3)$$

$$\Delta_{\text{grad}} = \left| \|\nabla_{\theta} L(P)\|_2 - \|\nabla_{\hat{\theta}} L(\hat{P})\|_2 \right|, \quad (4)$$

$$\Delta_{\text{param}} = \frac{\|\Delta\theta - \Delta\hat{\theta}\|_2}{\|\Delta\theta\|_2 + \epsilon}. \quad (5)$$

Here $\Delta\theta$ and $\Delta\hat{\theta}$ denote source and target parameter updates, and ϵ prevents division by zero. Figure 4 divides each difference by its detection threshold. The main migration comparison uses the task-specific tensor comparisons defined above.

B EVIDENCE HANDOFF AND AGENT INTEGRATION

The component study shares the main collection, initial translations, acceptance checks, and cumulative budgets. Continuing the original conversation retains its message roles and reasoning history when the agent begins editing. The independent handoff supplies the Verifier’s evidence to a separate repair conversation. Removing prior repair conversation restarts each subsequent repair attempt from the initial investigation and latest observation, while retaining the current files and cumulative budget. Removing progress reminders preserves the call and submission limits.

Table 6: Complete component and host-integration comparison on the common MindSpore collection. Token totals include initial translation and all subsequent calls.

Setting	Accepted	Total tokens (millions)
LaDiM	50/50	5.159
Continuous investigation and repair conversation	49/50	5.603
Without prior repair conversation	50/50	4.566
Without progress reminders	50/50	4.562
MatchFixAgent	50/50	12.097
MatchFixAgent with independent investigation	50/50	19.480
SWE-agent	44/50 [†]	20.831
SWE-agent with independent investigation	43/50	36.547

[†] Includes one task with unavailable final acceptance, which remains in the denominator and cost totals. Each condition uses the input reuse described in Appendix A.1.

LaDiM accepts all 50 identifiers, while the continuous conversation accepts 49. Removing prior repair conversation or progress reminders preserves full acceptance and uses fewer tokens. Twenty distinct initial translations are already accepted, and only two of the nine faulty pairs require a second repair submission under the complete method. Most cases therefore finish before a later repair attempt can use earlier repair conversation.

The natural translation component study uses the same ten supplied candidates as Table 3. Its reference configuration includes examples of the edit format and corrective feedback for malformed edits. Each condition changes the named component while retaining the candidates, workspace access, acceptance checks, and repair budgets. Removing this format assistance gives the LaDiM configuration reported in the natural fault comparison.

The host-integration conditions prepend the same read-only investigation to SWE-agent or MatchFixAgent. Investigation calls share the host’s total call, token, and time limits. The native host

Table 7: Effect of repair context and editing assistance on natural translation faults. Tokens cover all ten repair runs.

Condition	Initially incorrect programs repaired	Initially correct programs preserved	Repair tokens (millions)	Final accepted
Independent evidence handoff	4/5	5/5	15.548	9/10
Continuous conversation	3/5	5/5	14.516	8/10
Without repair history	0/5	5/5	10.852	5/10
Without progress reminders	2/5	5/5	17.325	7/10
Without editing format assistance	4/5	5/5	13.207	9/10

retains its own history and repair policy. MatchFixAgent with investigation retains 50/50 at a higher token total. SWE-agent with investigation accepts 43/50, compared with 44/50 and one interrupted task for its native condition. Across distinct inputs, the added investigation increases SWE-agent’s repair count from seven to eight among nine initially faulty programs. It also leaves four initially accepted programs failing at the end. Table 6 records both the repair and preservation consequences in final acceptance.

C ADDITIONAL REPAIR COLLECTIONS

We also evaluate supplied faulty target programs under their respective acceptance criteria. A controlled MindSpore collection contains 50 injected faults with task-specific acceptance checks. LaDiM accepts 45/50 within four submissions, compared with 29/50 for SWE-agent and 35/50 for a direct repair agent using shared tools. LaDiM’s cumulative accepted counts are 29, 41, and 45 at one, two, and four submissions. Total token use is 39,982,844, 42,579,064, and 12,459,013 for LaDiM, SWE-agent, and direct repair, respectively. The source and target inputs required by MatchFixAgent are supplied in the paired collection below.

In a separate collection of twelve source and target repair tasks, LaDiM accepts 12/12 and Match-FixAgent accepts 11/12. They use 6,642,603 and 14,505,833 tokens, respectively. Both receive the complete source, candidate, task specification, and target library. Each repair collection uses its specified editable code scope and acceptance checks.

D NATURAL TRANSLATION REPAIR PROTOCOL

The ten supplied translations contain five initially faulty programs and five that already pass the checks. Each method receives the same source, candidate, task requirements, and supporting library. The candidate and library may be edited. The source and acceptance criteria remain fixed. Target forward computation and differentiation execute in MindSpore through `torch4ms`, followed by PyTorch optimizer updates on its parameters. The comparison uses matching inputs and initial parameters with three-seed confirmation. Loss absolute difference, gradient norm absolute difference, and relative parameter update thresholds are 0.02, 0.05, and 0.03, with an additional gradient vector L2 threshold of 0.05 and the specified layer and parameter checks.

Table 3 counts every call for each input, including failed repairs and investigation of initially accepted programs. LaDiM uses 193 calls, direct repair uses 120, SWE-agent uses 386, and Match-FixAgent uses 265. LaDiM uses 13,206,863 tokens.

The LSTM repair in Section 4.3 adds a MindSpore recurrence and registers it in the supporting library, then corrects the tensor-indexing dispatch. On the final public check, loss absolute difference is 2.38×10^{-7} , gradient vector L2 difference is 4.19×10^{-7} , and relative parameter update difference is 8.11×10^{-7} . The example uses 32 model calls.

E REPOSITORY PROTOCOLS AND DETAILED RESULTS

All methods receive byte-identical source repositories, common initial translations, task requirements, and acceptance procedures. The time series translation has three generation calls totaling

163,661 tokens. The recommendation translation uses 20 calls totaling 368,368 tokens. Generation proceeds from the public import structure and includes failed or truncated attempts. Each method’s reported total contains one complete copy of this generation cost together with every repair call. Table 8 separates input and output usage in repair.

Table 8: Complete costs for the repository comparison. Total tokens include repair input and output plus common initial translation.

Method	Repair calls	Submissions	Repair input tokens	Repair output tokens	Total tokens
<i>Time series</i>					
LaDiM	35	2	2,345,473	105,096	2,614,230
SWE-agent	80	1	7,984,797	51,367	8,199,825
MatchFixAgent	69	2	13,306,003	128,114	13,597,778
<i>Recommendation</i>					
LaDiM	80	4	8,147,846	295,032	8,811,246
SWE-agent	80	1	7,889,936	58,248	8,316,552
MatchFixAgent	80	1	8,215,327	95,385	8,679,080

The time series protocol compares the core model for three consecutive Adam steps from corresponding source parameters, with dropout disabled and input shape $4 \times 20 \times 1$. Preprocessing and prediction absolute tolerances are 10^{-9} and 10^{-4} . Loss absolute difference, gradient vector L2 difference, and relative parameter update tolerances are 0.02, 0.05, and 0.03. Learning rate behavior is checked around its schedule boundaries. The public evaluation executes the 100-epoch training workflow, notebook cells, and teaching snippets in their source order. Each entry point must produce the required predictions and plots and execute native differentiation. The notebook and snippets provide execution coverage, while the core model supplies paired numerical training measurements. All three methods pass 23 behavior checks per seed, giving 69/69 across the public and two confirmation seeds.

The recommendation protocol covers six model paths that execute in the source framework with three sequential Adam updates. Source exceptions and empty returns are checked against the corresponding source behavior. Both the numerical worker and training command invoke the candidate’s optimizer factory with learning rate 0.001. The source Adam settings use decay coefficients 0.9 and 0.999 and epsilon 10^{-8} . Representation absolute tolerance is 10^{-4} . Loss absolute/relative tolerances are 0.002/ 10^{-4} , gradient tolerances are 10^{-4} /0.005, and relative parameter update tolerance is 0.03. The evaluation includes all ten original tests and one epoch of the training command on 16 synthetic examples.

Table 9 reports the declared checks on the public seed. The 114 numerical comparisons comprise 108 checks over six paths, three steps, and six training signals, plus six inference retrieval checks. Twenty-four execution, step count, and native differentiation checks and seven source behavior checks bring the behavior-check total to 145. SWE-agent and MatchFixAgent each lack one retrieval measurement after reward model inference fails. That entry remains n/a in both the 114 and 145 denominators. MatchFixAgent passes 0/10 original tests: an unmigrated PyTorch import in its history-encoder test prevents collection under the native MindSpore requirement. No method reaches the two confirmation seeds on this repository.

F REPOSITORY CONTEXT AND EVIDENCE

The repository extension separates the version of its dependency plan from the version of the candidate files. Every recorded test stores the candidate version on which it ran. An actual change to candidate code makes earlier measurements stale, and a new result replaces the preceding result for the same test even when the new test fails. Work-unit checkpoints additionally depend on their declared file scopes, test selections, and prerequisite units. A checkpoint without a selected test records syntax validation only. The final acceptance decision uses the complete repository evaluation.

Full tool observations, file reads, and conversations are archived outside the editable workspace. Records carry content hashes and can be retrieved by identifier, JSON pointer, and character range.

Table 9: Recommendation repository checks passed on the public seed. The numerical checks are a subset of behavior checks, and the six training signal rows break down the training checks. One unmeasured retrieval check remains in each baseline’s denominator.

Check	LaDiM	SWE-agent	MatchFixAgent
Behavior checks passed	130/145	104/145	129/145
Numerical checks	99/114	74/114	99/114
Training checks	93/108	69/108	94/108
Inference retrieval	6/6	5/6	5/6
User representations	18/18	16/18	18/18
Item representations	6/18	6/18	6/18
Forward loss	18/18	18/18	18/18
Training return loss	18/18	0/18	18/18
Gradients	18/18	17/18	18/18
Parameter updates	15/18	12/18	16/18
Original tests	10/10	10/10	0/10
Training command	1/1	1/1	1/1
File coverage	1/1	1/1	0/1
Documentation and dependencies	1/1	1/1	0/1

When the context is rebuilt, it retains the current dependency plan and generation status, the complete structured hypotheses and next steps, the latest measurements, and the last tool batch. Cached code consists only of content returned by earlier reads, with matching current file hashes and relevance to the active unit. The cache contains at most 120,000 characters. Reconstruction is triggered by an independent role handoff, an actual work-unit switch, or messages exceeding 75% of the configured context capacity. Ordinary stage continuation and the first selection of a work unit retain the existing conversation.

For the repository experiments, diagnosis allows up to six calls and 64,000 output tokens. The remaining repair calls and output budget are allocated over the remaining external submissions, with at most 16 calls in the first repair stage. The cumulative limits apply to the whole repository.